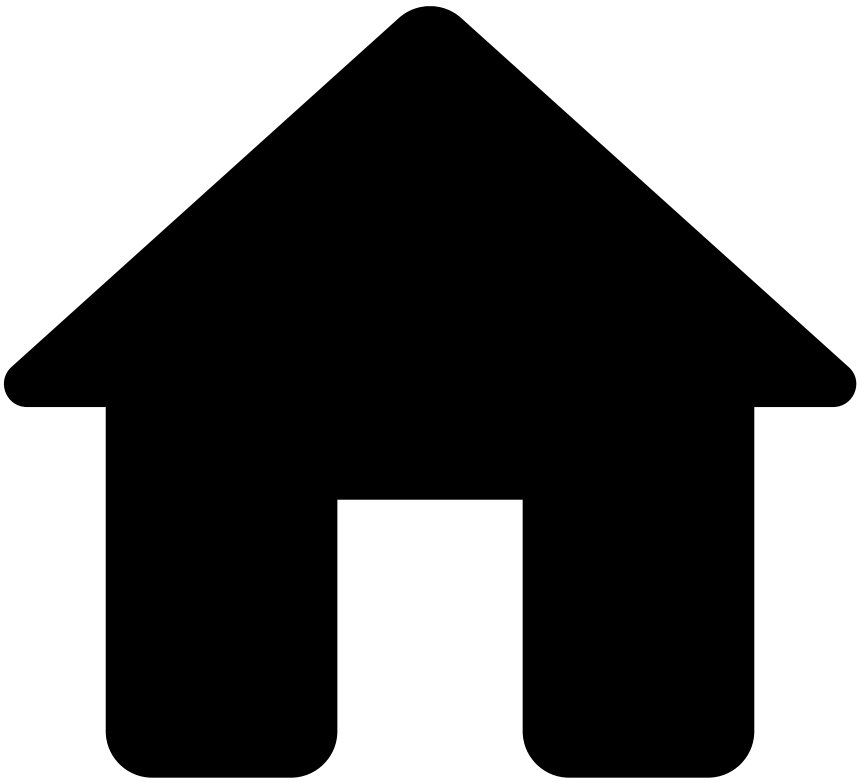


Designing a workflow

Classifying real-time events comprises several stages. For example, fraud detection is a task comprised of several steps with the aim of deciding whether or not an event represents a fraudulent transaction. Thus, Pulse lets you group these processing steps into a workflow.

Workflows represent the life cycle of an event through the system. They define how information is passed on between components that may filter and/or transform the event.

A **Workflow** consists of processing steps (elements) and links between them. Each workflow element processes or transforms the data as soon it receives data. Workflow elements can be seen as components that process data with the ability to apply operations such as feature extraction, model scoring, decision/detection rules, etc.

The following image depicts a **Workflow** where input data is processed with a Plan to generate features, scored with a Model, and evaluated by a block of Rules.

When designing a **Workflow** to process events in real time, the following activities are involved when adding and configuring elements:

- Configuring outcomes
- Adding input adapters
- Adding plans
- Adding models
- Adding rules
- Adding custom services

Workflow patternsâ□□

Workflows give you a lot of flexibility on how you can compose elements and define the path that events should follow. Check to following sections on some patterns you can use when designing a workflow.

Branchingâ□□

Pulse allows you to configure your workflow path in a way that there are multiple branches that can process each event. A common scenario is to apply different operations based on the transaction's country.

This can be achieved by adding a filter to each of the branching service's links, as depicted in the following image. Note that the expression for the filter can make use of any **Workflow** schema field, and must be a stateless PQL expression. Additionally, UDFs are not supported.

Parallel branching

Pulse does not execute separate workflow branches in parallel. Thus, although branching is possible, a real-time event will always follow a single linear path from input until the workflow terminates.

You must always make sure that when you branch the workflow, all possible paths are mutually exclusive. This means that **an event must only be routed to exactly one of the possible paths**.

The following examples show how to branch a workflow into two paths without leading to unsupported behavior:

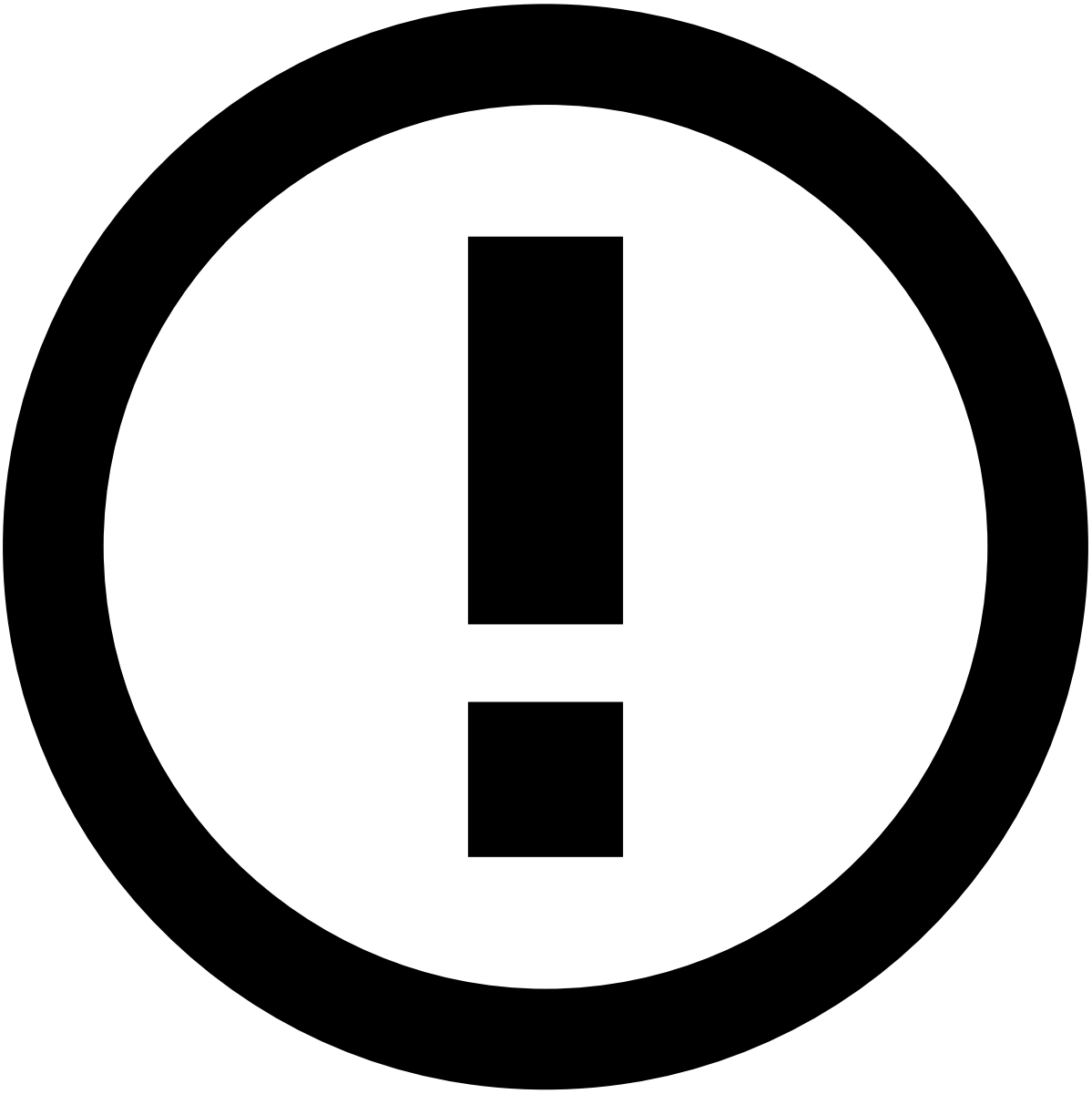
- Link A filters by "amount > 2000" and Link B filters by "amount <= 2000"

- Link A filters by "location == USA" and Link B filters by "location != USA"

If do not take these precautions there is more than one branch where the filter condition is *true*, than Pulse will route the event to the first one that evaluates as *true*, and write an exception to the log.

When joining branches into a single path you must make sure that all incoming paths carry events with the same **Schema**, as shown in the following diagram.

Since Pulse never executes both branches in parallel, once an event arrives at the Plan Service, it must have the same schema regardless of the original path it took.



Deployable Plans

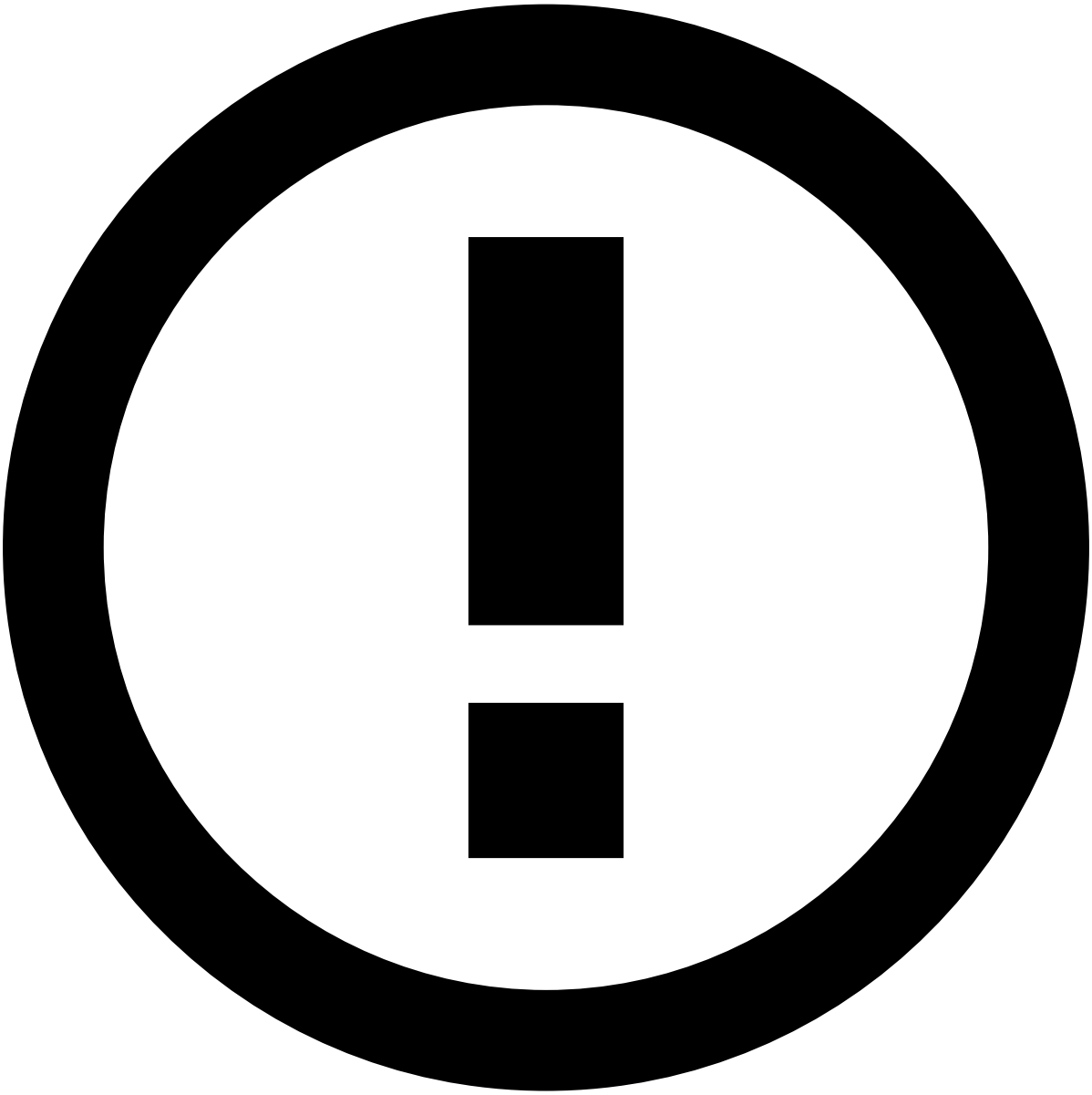
Deployable Plans only allow for a single Data Source to be configured as input.

Therefore, you must ensure that if you perform data transformations in any of the branches when an event arrives at the join point it still has a data schema that is compatible with the expected input for the **Plan Service**.

Branching with Tenantsâ

Multi-Tenancy allows managing multiple Tenants in the same installation. The resources that support **Multi-Tenancy** are **Rules** and **Lists**. You can, for example, apply the same **Rule** to all the Tenants, or different **Rules** to different Tenants with just a few configuration steps.

However, imagine that you would like to have different **Plans** or different **Models** applied to different Tenants. This is still possible using **Branching with Tenants** as an alternative approach to the native **Multi-Tenancy** solution, as long as the different configured **Plans** and **Models** expect the same input and produce the same output Schema, so that subsequent Workflow steps, such as **Rules**, can be then applied to these non Multi-Tenanted resources. This configuration can only be performed with special permissions, and thus, is at the level of the Landlord (*i.e.* the Tenants cannot use **Branching with Tenants**).

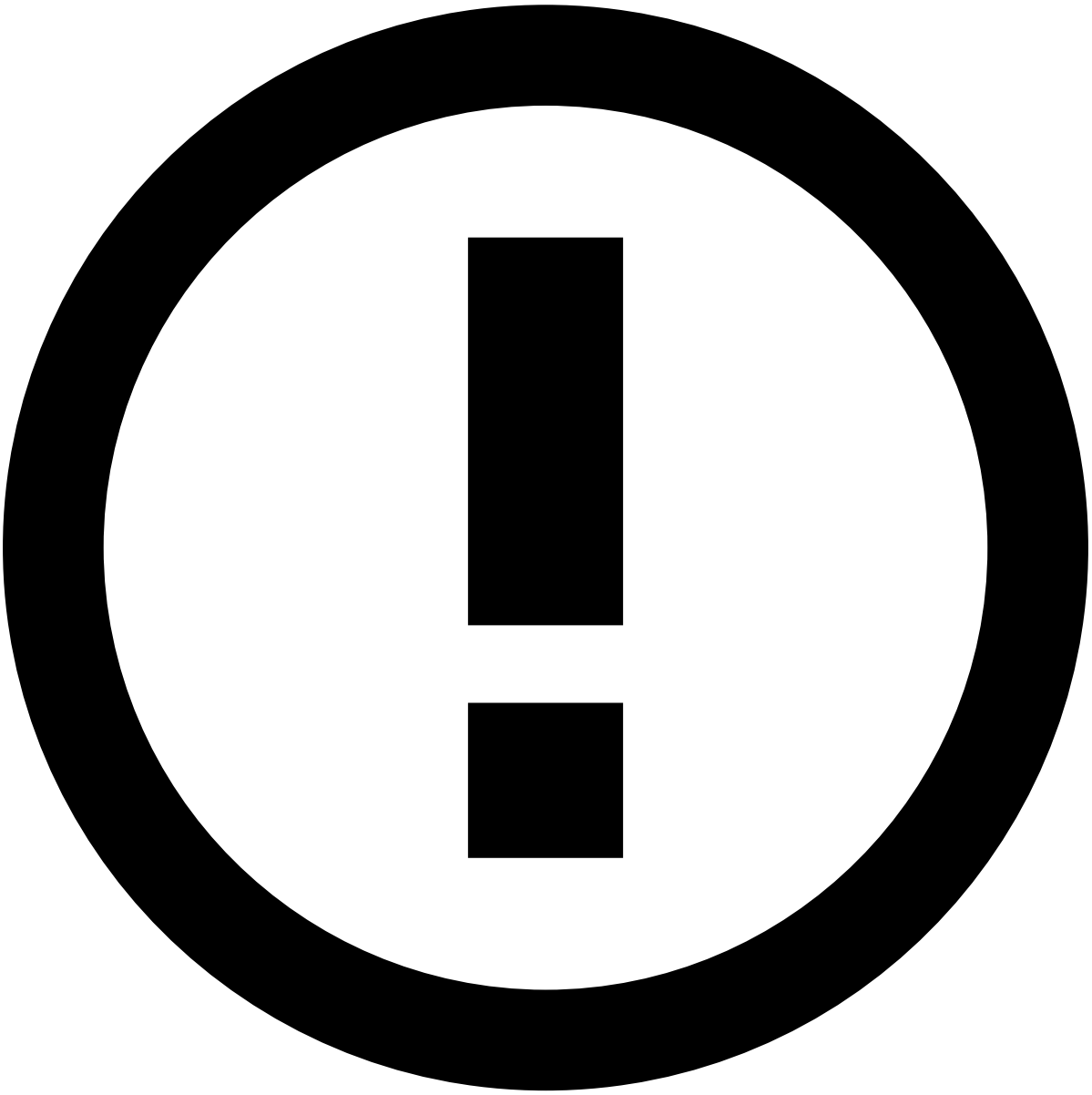


Branching with Tenants

Note that this alternative approach to the **Multi-Tenancy** solution is viable because normally the same **Plan** and **Model** is used with all the Tenants and only in exceptional cases, a few Models or Plans are segregated by tenant.

However, this is not the case for **Rules.** The normal scenario is to have many Rules applied to all the Tenants or many Rules segregated by Tenant. Thus, **Branching with Tenants** is not a practical solution to be used with **Rules Projects**.

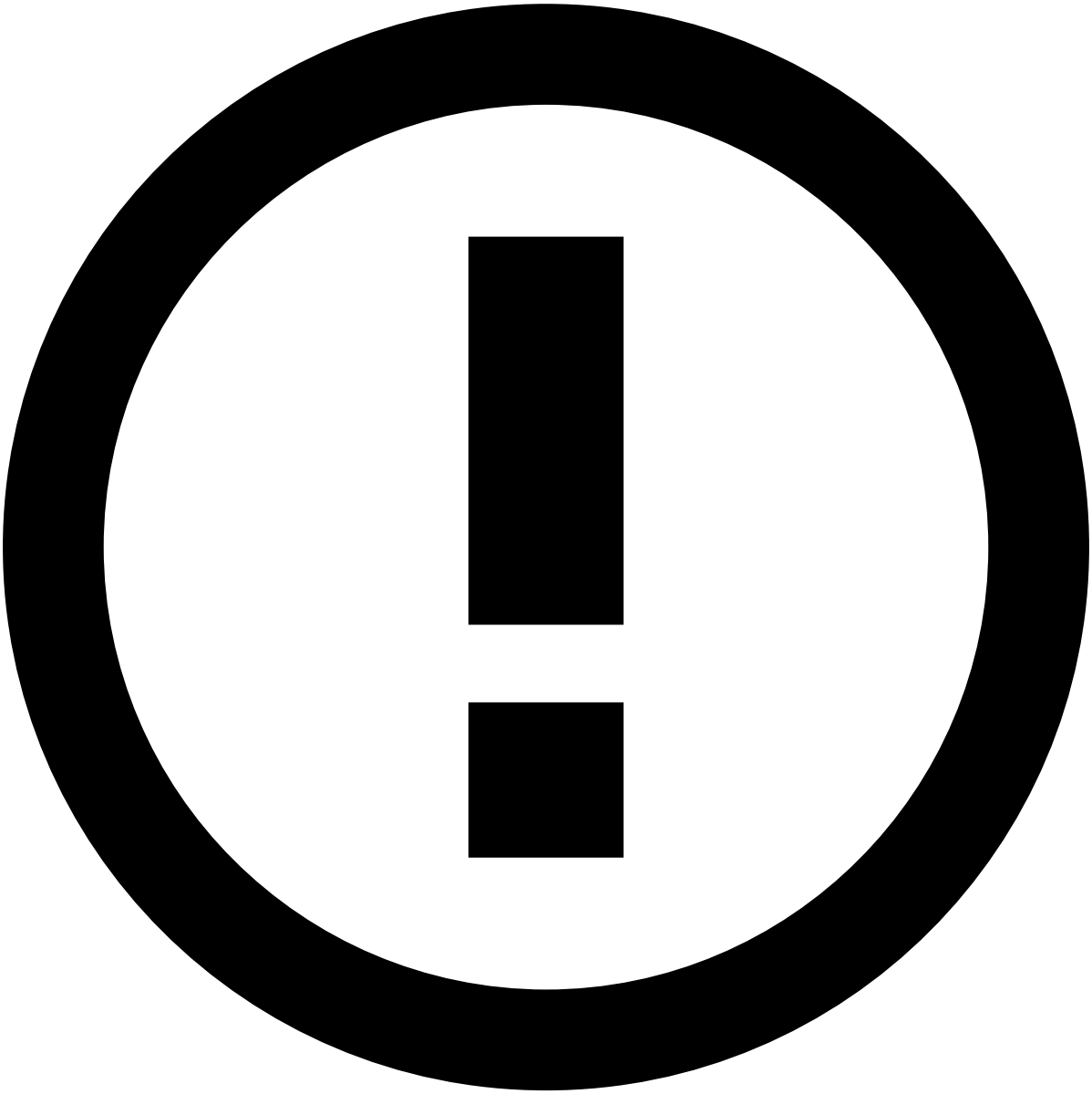
Considering the scenario of two different **Plans**, this can be achieved with **Workflow** configuration by adding a filter to the branches that connect the **Input Adapter** to both **Plans**, as displayed in the following image.



Adapt Schema

Note that the Schema must have a field identifying the Tenant so that it can be used to configure the filter and allow a real-time event to follow the right path.

Considering the scenario of two different **Models**, this can be achieved with **Workflow** configuration by adding a filter to the branches that connect the **Plan** to both **Models**, as displayed in the following image.



Adapt Schema

Note that the Schema has to have a field identifying the Tenant so that it can be used to configure the filter and allow a real-time event to follow the right path.

Applying different models to the same eventâ□□

Depending on the scenario you might need to get the contribution of several models before reaching a final decision, or just running events through different models to select the best result.

Contributing to an outcomeâ□□

To take into account the contribution of several models to the same outcome (for example *fraud* and *fraud_score*), you must run events through different **Model Services** in sequence.

This pattern is very similar to a regular **Workflow** with a **Scoring Service** followed by a **Rules Service**. The contributions of the several models are simply reconciled using the configured weights for each **Scoring Service**.

The following image displays an example where events are scored by two models in sequence, contributing to the same decision. The final score takes the weights of each model into account.

Selecting the best modelâ□□

In scenarios where you want to run an event through several models and make your final decision based on the best absolute result, your **Workflow** should:

- Include a **Scoring Service** for each model.
- Configure separate Outcomes that represent the partial decisions of each model.
- Include a Custom Workflow Service that takes the outputs and outcomes of all **Scoring Services** and produces the final decision.

The following image displays a workflow where events are scored by two models in sequence, each one contributing to a different outcome, and a **Custom Service** to perform the final decision.

Using detection and decision rulesâ□□

It is common to route transactions through different sets of Rules at different points in the **Workflow**. Typically, you may need to add the following types of **Rules**:

- **Detection Rules** Used in conjunction with Lists, let you immediately identify events that match an entry in a list. For example, blacklisting and whitelisting.
- **Decision Rules** Used in conjunction with Models, weigh in on the final Outcomes and scores for an event after it has been transformed and scored by a model.

The following image shows how such a workflow will look like.

Edit Workflow pageâ□□

The **Edit Workflow page** provides a design area where you can model your Workflow through which your application will process real-time events.

The following image displays the **Edit Workflow page:**

The Edit Workflow page contains the following areas:

1. **Configurations**
   Depending on the active tab, **Workflow** or **Advanced**, contains the fields that you need to configure for your workflow. The most important fields are:
   - **Origin Schema**
     The Schema definition of the input data expected by the workflow. Select an existing Data Source to automatically generate the respective Schema.
   - **Outcomes**
     outcomes to add decision and score fields to events, and effectively define to which decisions will workflow **Services** be contributing to. For example, fraud/not-fraud and the fraud score.
   - **Actions**
     The actions that workflow **Services** can generate. For example, a **Rules Service** may generate a **Block** action for transactions identified as fraud.
   - **Action Listeners**
     Custom Services that execute custom logic to handle specific **Actions** whenever a **Workflow** element raises them.
2. **Services**
   The available elements you can use to compose your workflow. The following elements are available:
   - **Input Adapter**
     Applies a Custom Service to consume external data and inject it into the workflow.
   - **Plan**
     Deploys data transformations, exactly as modeled during the Data Science stage. Typically, to generate features out of raw input data before feeding it to a Machine Learning Model.

- **Scoring**
  Runs the events through a Machine Learning Model, exactly as parameterized and trained during the Data Science stage. Contributes to an outcome and the corresponding score.
- **Rules**
  Runs the events through a Rules block, and similarly to the **Scoring Service**, also contributes to an outcome and the corresponding score.
- **Custom Workflow Service**
  Deploys a Custom Service to perform specific operations on events, that Pulse does not implement out-of-the-box.

3. **Diagram**
   Canvas to where you must drag the **Services** and visually design your workflow.
   Edit services to define their main configurations such as Data Science snapshots, verify their input and output schemas and their mapping with the event schema, and properties.
   Drag and drop the handles to connect two services in the workflow. **Double-click** an edge connecting two services to edit a filter condition, for example, only allowing events from a specific country to advance through this edge. This can be used to route these events to a different model for scoring.